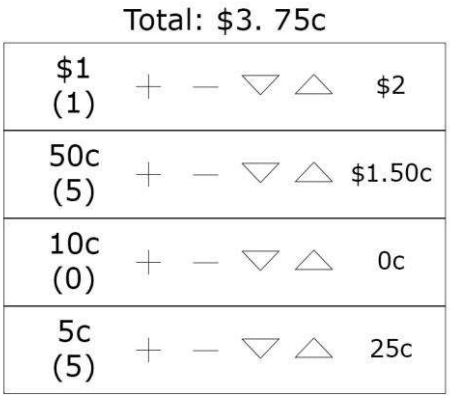
Currency App (A)

This portfolio exercise covers…

- Naming conventions.
- Model-View-Controller.
- Coding rules.
- Code documentation.

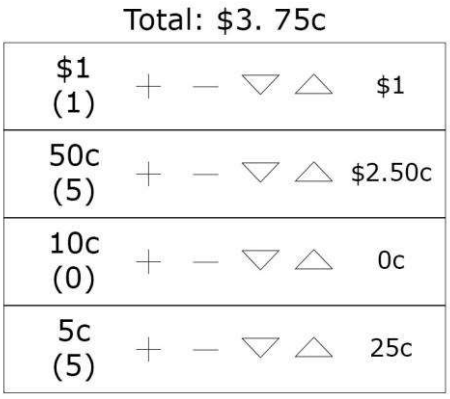
Brief

Create an interface that looks something like this:



Do not worry about perfect alignment and recreating this design but make sure it at least includes the elements shown including a total label at the top and 4 sections for Australian coins from 5c through to $1 (excluding 20c). In each section have two or three labels to show the coin, how many you have and the total of each. Four buttons; an Add, Subtract, convert up and convert down arrow.

The purpose of the app is to track how many coins someone has and be able to convert coins from one to the other from $1 down to 5c. So, if I pressed the convert down button on the $1 section it would remove $1 and add two 50c's to the 50c area. In the example above after pressing the convert down on 1$ it would look like this:



The buttons should behave as expected to add, remove coins and convert between each coin. If there are not enough coins to perform a conversion the convert button should be greyed out.

The Model

Your first step should be to create the data model - the class that will store everything the app needs to know or remember about the user's data. It also needs to manipulate that data (searching, sorting, saving, loading, converting, etc). For our Money Tracker app, the data model class should store:

Amounts of:

- $1 coins
- 50c coins
- 10c coins
- 5c coins

The data model should also have corresponding methods to handle any actions or interactions with these coins (adding, subtracting and converting)

The App

Create the Data model for the application, Remember:

- You should abide by the C# naming conventions.
- Code should be self-documenting.
- Code should not be unnecessarily complex or clever. Simple code is readable code.
- All classes and methods should have a short comment explaining what they do.
- Every class should be in its own file.
- Each class does one job.
- Every class should be a model class or controller class with no overlap. (You can also have view classes, in theory, but that side of things is being handled by the XAML for us.)

Once the data model is completed add the UI elements and connect these together.

These rules are a great guide that you should follow in all your code. Its great practice for you to get into the habit and ensure you are creating readable and maintainable code. As a result, these expectations should be followed in <u>All</u> submissions going forward.

<u>Bonus: Add the 20c in-between 50c and 10c and create a working solution for converting up and down like the rest.</u>

Total: $3. 75c

$1 (2)	+	—	▽	△	$2
50c (3)	+	—	▽	△	$1.50c
20c (0)	+	—	▽	△	0c
10c (0)	+	—	▽	△	0c
5c (5)	+	—	▽	△	25c